Evaluation Event II/III/Final - *Agent Account Name*

Daria Stetsenko (*daria.stetsenko@uzh.ch*)
Zahraa Zaiour (*zahraa.zaiour@uzh.ch*)

November 18, 2025

1 Introduction

Our agent implements a hybrid approach to movie knowledge graph question answering, combining pattern-based analysis, embedding-based retrieval, LLM-assisted SPARQL generation, and graph-based recommendations. The system supports factual queries about movies, directors, actors, genres, and other movie-related information using a Wikidata-based knowledge graph, as well as personalized movie recommendations. Key features include robust entity extraction, flexible query pattern recognition, a fallback strategy using embeddings for complex queries, and a graph-based recommender system that leverages both knowledge graph structure and rating data. The agent employs both traditional rule-based methods and modern neural approaches, ensuring reliable answers while handling natural language variations.

2 Capabilities

Our agent supports the following question types:

- Factual Questions
  The agent processes factual queries using a three-stage pipeline: 1) Query pattern analysis using a transformer classifier, 2) Entity extraction with spaCy NER and custom pattern matching, and 3) Dynamic SPARQL generation with LLM assistance. The system supports forward queries (movie → property), reverse queries (person → movies), and verification queries (relationship checks).

- Embedding Questions
  For embedding-based queries, we use a TransE-based approach combining sentence transformers for query embedding and learned alignments to the knowledge graph embedding space. The system can handle both direct similarity search and structured relation-based queries through embedding composition.

- Recommendation Questions
  The agent handles recommendation queries through a graph-based recommender system that combines Wikidata structure with rating data. The system supports: 1) Theme-based recommendations (e.g., Disney-style animation, slasher horror, literary drama), 2) Director/actor/writer filmographies, 3) Award-based lists, and 4) Top/lowest-rated queries. The recommender uses seed movie detection, intent classification, theme inference from genres and years, and rating-aware ranking with domain-specific filters.

- Hybrid Processing
  The agent implements a fallback strategy where factual queries that fail pattern matching can be processed through the embedding pipeline, ensuring robust handling of edge cases and complex queries.

3 Adopted Methods

- DeepSeek Coder (1.3B)[1]
  Used for natural language to SPARQL translation through few-shot prompting. The lightweight model enables fast local inference while maintaining good query generation quality.

- sentence-transformers[2]
  Specifically the all-MiniLM-L6-v2 model for embedding natural language queries. Selected for its efficiency and strong semantic understanding capabilities.

- spaCy[3]
  Employed for Named Entity Recognition in the entity extraction pipeline, particularly effective for identifying person names and movie titles.

- TransE[4]
  Knowledge graph embeddings for semantic similarity search and relation-based reasoning. The model encodes entities and relations in a shared vector space.

- RDFLib[5]
  Core library for handling RDF graph operations and SPARQL query execution against the movie knowledge graph.

[1] https://github.com/deepseek-ai/DeepSeek-Coder
[2] https://www.sbert.net/
[3] https://spacy.io/
[4] https://papers.nips.cc/paper/2013/hash/1cecc7a77928ca8133fa24680a88d2f9-Abstract.html
[5] https://rdflib.readthedocs.io/

- llama-cpp-python[6]
  Efficient inference engine for running the DeepSeek Coder model locally, enabling fast SPARQL generation.

- Graph-Based Recommender System
  Implemented as a MovieGraphIndex class that builds a movie-centric index over the Wikidata graph plus rating CSVs. The system combines graph structure (genres, directors, writers, actors, awards) with rating statistics to answer recommendation queries. It supports award-based lists, top/lowest-rated queries, director/writer/actor filmographies, and theme-based recommendations. Theme inference uses genre analysis and release years to identify patterns like Disney-style animation, slasher horror, or literary drama. The recommender employs weighted scoring combining genre overlap, theme-specific bonuses, and ratings, with seed exclusion and domain-specific filters.

4 Examples

- Factual Questions
  For the question "Who directed The Matrix?", the agent: 1) Identifies a forward query pattern with director relation 2) Extracts "The Matrix" as the movie entity using case-insensitive matching 3) Generates a SPARQL query with proper prefixes and filters 4) Executes against the knowledge graph to retrieve the director

- Embedding Questions
  Given "Please answer with embeddings: What genre is Inception?": 1) Embeds the query using sentence-transformers 2) Aligns to TransE space using learned projection 3) Uses TransE composition (movie + genre relation) 4) Finds nearest genre entities in the embedding space 5) Validates results against expected entity types

- Recommendation Questions
  For "Given that I like The Lion King, Pocahontas, and Beauty and the Beast, can you recommend some movies?", the system detects Disney-style animation themes through genre analysis (animated, family, fantasy) and recommends similar family films while excluding adult animation. For "Recommend movies like Nightmare on Elm Street, Friday the 13th, and Halloween", the system identifies classic slasher horror from the 1970s-1980s and returns horror films from similar decades ranked by genre overlap and rating. For "movies directed by Christopher Nolan", the director index retrieves his complete filmography ranked by rating, excluding any seed movies mentioned in the query. The system similarly handles actor queries, Shakespeare tragedy recommendations, and top/lowest-rated lists through specialized detection and ranking strategies.

[6] `https://github.com/abetlen/llama-cpp-python`

- Hybrid Processing
  For complex queries like "From what country is Parasite?": 1) Attempts pattern matching first 2) Falls back to embedding similarity if needed 3) Validates results using entity type constraints 4) Returns formatted natural language response

5 Additional Features

Key improvements in our implementation include:

- Robust entity extraction with multi-strategy approach:

```
# Priority order for entity extraction:
1. Quoted text extraction
2. spaCy NER
3. Capitalized spans
4. Pattern-based matching
```

- Smart case handling for movie titles:

```
# Proper English title case:
"the bridge on the river kwai" →
"The Bridge on the River Kwai"
```

- LLM-first SPARQL generation with template fallback:

```
1. Try DeepSeek Coder with few-shot examples
2. Fallback to templates if LLM fails
3. Validate and clean generated queries
```

- Recommender system with theme detection:

```
# Theme inference pipeline:
1. Extract seed movies from query
2. Detect intent (director/actor/theme/award)
3. Infer theme profile from genres + years
4. Apply domain-specific filters
5. Rank by weighted score (genre + rating)
```

6 Conclusions

Future improvements planned:

- Enhanced support for temporal queries and date handling
- Integration of multimedia content for movie-related images
- Expansion of the knowledge graph with additional movie data sources
- Implementation of a query caching system for improved response times
- Learning recommender scoring weights from user interaction logs
- Content-based recommendations using embedding proximity

References

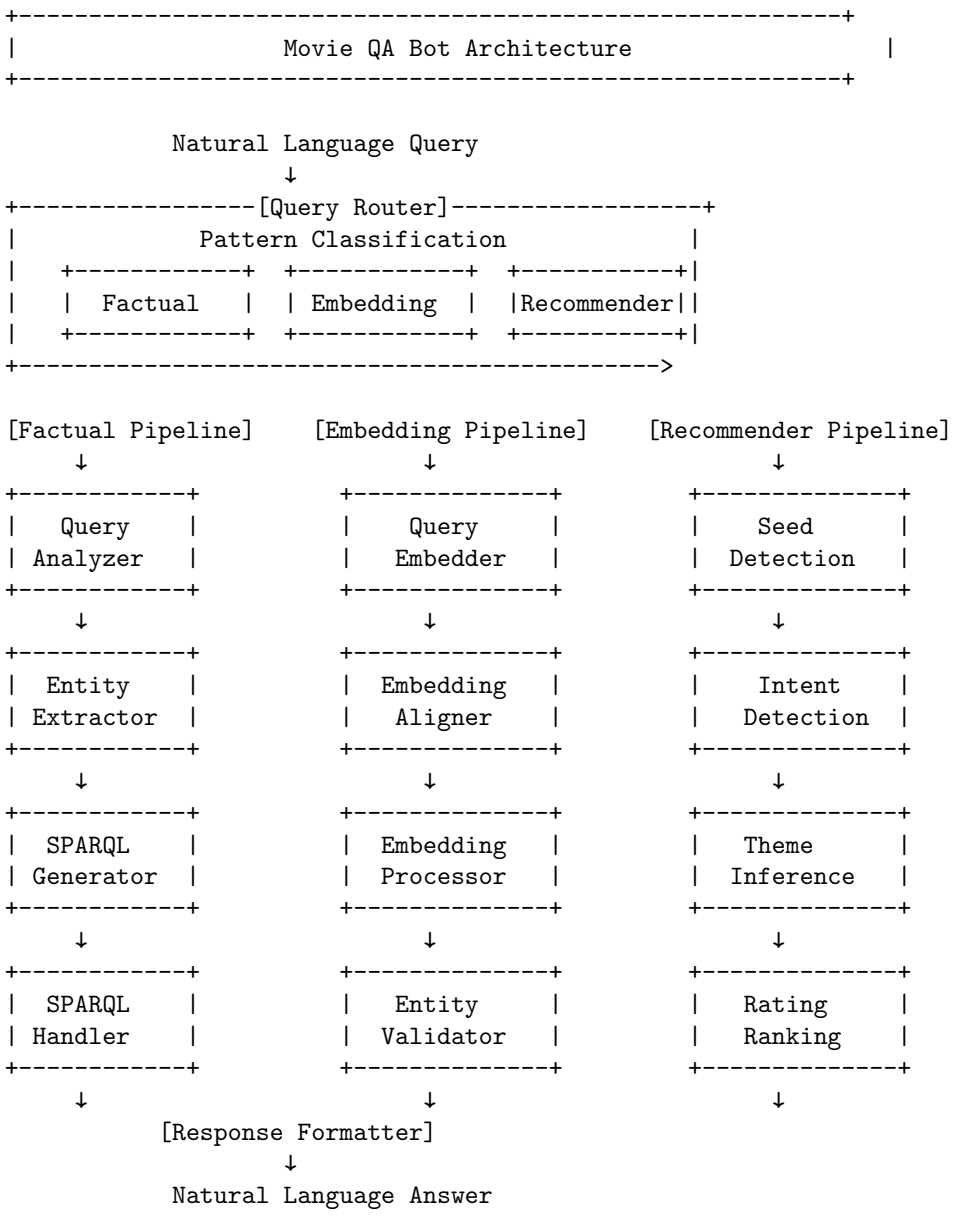

Figure 1: Architecture overview showing the main components: Query Analyzer, Entity Extractor, SPARQL Generator, Embedding Processor, and Recommender System